

Question 1: Implement a Stack using 1 Queue (User Input)

Code

```
#include <stdio.h>

#include <stdlib.h>

#define MAX 100

typedef struct {
    int arr[MAX];
    int front, rear;
} Queue;

void initQueue(Queue *q) {
    q->front = -1;
    q->rear = -1;
}

int isEmpty(Queue *q) {
    return (q->front == -1 || q->front > q->rear);
}

int isFull(Queue *q) {
    return q->rear == MAX - 1;
}

int size(Queue *q) {
    if (isEmpty(q)) return 0;
    return q->rear - q->front + 1;
}

void enqueue(Queue *q, int val) {
    if (isFull(q)) {
        printf("Queue Overflow\n");
        return;
    }
    if (isEmpty(q)) {
        q->front = 0;
        q->rear = 0;
    } else {
        q->rear++;
    }
}
```

```

    }

    q->arr[q->rear] = val;
}

int dequeue(Queue *q) {
    if (!isEmpty(q)) {
        int val = q->arr[q->front++];
        if (q->front > q->rear) {
            q->front = -1;
            q->rear = -1;
        }
        return val;
    }
    return -1;
}

typedef struct {
    Queue q;
} Stack;

void push(Stack *s, int val) {
    enqueue(&s->q, val);
}

int pop(Stack *s) {
    if (isEmpty(&s->q)) {
        printf("Stack Underflow\n");
        return -1;
    }

    int current_size = size(&s->q);
    for (int i = 0; i < current_size - 1; i++) {
        enqueue(&s->q, dequeue(&s->q));
    }

    return dequeue(&s->q);
}

int main() {
    Stack s;

```

```

initQueue(&s.q);

int choice, val;

while (1) {
    printf("\n1. Push 2. Pop 3. Exit\nEnter choice: ");
    if (scanf("%d", &choice) != 1 || choice == 3) break;
    if (choice == 1) {
        printf("Enter value to push: ");
        scanf("%d", &val);
        push(&s, val);
        printf("Pushed %d\n", val);
    } else if (choice == 2) {
        val = pop(&s);
        if (val != -1) printf("Popped: %d\n", val);
    }
}

return 0;
}

```

Output:

1. Push 2. Pop 3. Exit

Enter choice: 1

Enter value to push: 10

Pushed 10

1. Push 2. Pop 3. Exit

Enter choice: 1

Enter value to push: 20

Pushed 20

1. Push 2. Pop 3. Exit

Enter choice: 2

Popped: 20

1. Push 2. Pop 3. Exit

Enter choice: 3

Question 2: Implement a Queue using 2 Stacks

Code

```
#include <stdio.h>

#include <stdlib.h>

#define MAX 100

typedef struct {
    int arr[MAX];
    int top;
} Stack;

void initStack(Stack *s) {
    s->top = -1;
}

int isEmpty(Stack *s) {
    return s->top == -1;
}

int isFull(Stack *s) {
    return s->top == MAX - 1;
}

void push(Stack *s, int val) {
    if (!isFull(s)) {
        s->arr[++(s->top)] = val;
    }
}

int pop(Stack *s) {
    if (!isEmpty(s)) {
        return s->arr[(s->top)--];
    }
}
```

```

    return -1;
}

typedef struct {
    Stack s1, s2;
} Queue;

void enqueue(Queue *q, int val) {
    if (isFull(&q->s1)) {
        printf("Queue Overflow\n");
        return;
    }
    push(&q->s1, val);
}

int dequeue(Queue *q) {
    if (isEmpty(&q->s1) && isEmpty(&q->s2)) {
        printf("Queue Underflow\n");
        return -1;
    }
    if (isEmpty(&q->s2)) {
        while (!isEmpty(&q->s1)) {
            push(&q->s2, pop(&q->s1));
        }
    }
    return pop(&q->s2);
}

int main() {
    Queue q;
    initStack(&q.s1);
    initStack(&q.s2);
    int n, val;
    printf("Enter number of elements to enqueue: ");
    scanf("%d", &n);
    printf("Enter %d elements:\n", n);
    for (int i = 0; i < n; i++) {
        scanf("%d", &val);
    }
}

```

```
        enqueue(&q, val);
    }
    printf("\nDequeuing elements:\n");
    for (int i = 0; i < n; i++) {
        printf("Dequeued: %d\n", dequeue(&q));
    }
    return 0;
}
```

Output:

Enter number of elements to enqueue: 3

Enter 3 elements:

100 200 300

Dequeuing elements:

Dequeued: 100

Dequeued: 200

Dequeued: 300

Question 3: Manage History of Visited Web Pages using Stack

Code

```
#include <stdio.h>

#include <stdlib.h>

#include <string.h>

#define MAX 100

#define URL_LEN 100

typedef struct {

    char pages[MAX][URL_LEN];

    int top;

} WebHistory;

void initHistory(WebHistory *h) {

    h->top = -1;

}

void visitPage(WebHistory *h, const char *url) {

    if (h->top < MAX - 1) {

        h->top++;

        strcpy(h->pages[h->top], url);

        printf("Visited: %s\n", url);

    } else {

        printf("History Full\n");

    }

}

void goBack(WebHistory *h) {

    if (h->top <= 0) {

        printf("No previous page in history.\n");

    } else {

        h->top--;

        printf("Navigated back to: %s\n", h->pages[h->top]);

    }

}

void currentPage(WebHistory *h) {

    if (h->top != -1) {

        printf("Current Page: %s\n", h->pages[h->top]);

    }

}
```

```

    } else {
        printf("No pages visited yet.\n");
    }
}

int main() {
    WebHistory history;
    initHistory(&history);
    int choice;
    char url[URL_LEN];
    while (1) {
        printf("\n1. Visit Page  2. Go Back  3. View Current Page  4. Exit\nEnter choice: ");
        if (scanf("%d", &choice) != 1 || choice == 4) break;
        if (choice == 1) {
            printf("Enter URL: ");
            scanf("%s", url);
            visitPage(&history, url);
        } else if (choice == 2) {
            goBack(&history);
        } else if (choice == 3) {
            currentPage(&history);
        }
    }
    return 0;
}

```

Output:

1. Visit Page 2. Go Back 3. View Current Page 4. Exit

Enter choice: 1

Enter URL: google.com

Visited: google.com

1. Visit Page 2. Go Back 3. View Current Page 4. Exit

Question 4: Sort a Stack using a Temporary Stack

Code

```
#include <stdio.h>

#include <stdlib.h>

#define MAX 100

typedef struct {
    int arr[MAX];
    int top;
} Stack;

void initStack(Stack *s) {
    s->top = -1;
}

int isEmpty(Stack *s) {
    return s->top == -1;
}

void push(Stack *s, int val) {
    if (s->top < MAX - 1) {
        s->arr[++(s->top)] = val;
    }
}

int pop(Stack *s) {
    if (!isEmpty(s)) {
        return s->arr[(s->top)--];
    }
    return -1;
}

int peek(Stack *s) {
    if (!isEmpty(s)) {
        return s->arr[s->top];
    }
    return -1;
}

void sortStack(Stack *input) {
    Stack tmpStack;
```

```

initStack(&tmpStack);

while (!isEmpty(input)) {
    int temp = pop(input);
    while (!isEmpty(&tmpStack) && peek(&tmpStack) > temp) {
        push(input, pop(&tmpStack));
    }
    push(&tmpStack, temp);
}
while (!isEmpty(&tmpStack)) {
    push(input, pop(&tmpStack));
}
}

int main() {
    Stack s;
    initStack(&s);
    int n, val;
    printf("Enter number of elements in stack: ");
    scanf("%d", &n);
    printf("Enter %d elements:\n", n);
    for (int i = 0; i < n; i++) {
        scanf("%d", &val);
        push(&s, val);
    }
    sortStack(&s);
    printf("\nSorted Stack (Ascending top to bottom):\n");
    while (!isEmpty(&s)) {
        printf("%d ", pop(&s));
    }
    printf("\n");
    return 0;
}

```

Output:

Enter number of elements in stack: 5

Enter 5 elements:

34 3 31 98 92

Sorted Stack (Ascending top to bottom):

3 31 34 92 98

Question 5: Stock Span Problem using Stack

Code

```
#include <stdio.h>

#define MAX 100

typedef struct {
    int arr[MAX];
    int top;
} Stack;

void initStack(Stack *s) {
    s->top = -1;
}

int isEmpty(Stack *s) {
    return s->top == -1;
}

void push(Stack *s, int val) {
    s->arr[++(s->top)] = val;
}

int pop(Stack *s) {
    return s->arr[(s->top)--];
}

int peek(Stack *s) {
    return s->arr[s->top];
}

void calculateSpan(int price[], int n, int span[]) {
    Stack s;
    initStack(&s);
    span[0] = 1;
    push(&s, 0);
    for (int i = 1; i < n; i++) {
        while (!isEmpty(&s) && price[peek(&s)] <= price[i]) {
            pop(&s);
        }
        span[i] = isEmpty(&s) ? (i + 1) : (i - peek(&s));
        push(&s, i);
    }
}
```

```

    }
}

int main() {
    int n;

    printf("Enter number of daily price quotes: ");

    scanf("%d", &n);

    int price[MAX], span[MAX];

    printf("Enter prices:\n");

    for (int i = 0; i < n; i++) {
        scanf("%d", &price[i]);
    }

    calculateSpan(price, n, span);

    printf("\nStock Spans:\n");

    for (int i = 0; i < n; i++) {
        printf("%d ", span[i]);
    }

    printf("\n");

    return 0;
}

```

Output:

Enter number of daily price quotes: 7

Enter prices:

100 80 60 70 60 75 85

Stock Spans:

1 1 1 2 1 4 6

Question 6: Next Higher Frequency Element to the Right using Stack

Code

```
#include <stdio.h>

#define MAX 100

typedef struct {
    int arr[MAX];
    int top;
} Stack;

void initStack(Stack *s) {
    s->top = -1;
}

int isEmpty(Stack *s) {
    return s->top == -1;
}

void push(Stack *s, int val) {
    s->arr[++(s->top)] = val;
}

int pop(Stack *s) {
    return s->arr[(s->top)--];
}

int peek(Stack *s) {
    return s->arr[s->top];
}

void nextGreaterFrequency(int arr[], int n, int result[]) {
    int freq[MAX] = {0};
    for (int i = 0; i < n; i++) {
        freq[arr[i]]++;
    }

    Stack s;
    initStack(&s);
    for (int i = n - 1; i >= 0; i--) {
        while (!isEmpty(&s) && freq[arr[peek(&s)]] <= freq[arr[i]]) {
            pop(&s);
        }
    }
}
```

```

    if (isEmpty(&s)) {
        result[i] = -1;
    } else {
        result[i] = arr[peek(&s)];
    }
    push(&s, i);
}
}

int main() {
    int n;
    printf("Enter number of elements: ");
    scanf("%d", &n);
    int arr[MAX], result[MAX];
    printf("Enter elements:\n");
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }
    nextGreaterFrequency(arr, n, result);
    printf("\nNext Greater Frequency Elements:\n");
    for (int i = 0; i < n; i++) {
        printf("%d ", result[i]);
    }
    printf("\n");
    return 0;
}

```

Output:

Enter number of elements: 6

Enter elements:

2 1 1 3 2 1

Next Greater Frequency Elements:

1 -1 -1 2 1 -1

Question 7: Minimum Number Following Pattern ('I' and 'D') using Stack

Code

```
#include <stdio.h>

#include <string.h>

#define MAX 100

typedef struct {
    int arr[MAX];
    int top;
} Stack;

void initStack(Stack *s) {
    s->top = -1;
}

int isEmpty(Stack *s) {
    return s->top == -1;
}

void push(Stack *s, int val) {
    s->arr[++(s->top)] = val;
}

int pop(Stack *s) {
    return s->arr[(s->top)--];
}

void printMinNumberForPattern(const char *pattern) {
    Stack s;
    initStack(&s);
    int n = strlen(pattern);
    printf("Resulting minimum number: ");
    for (int i = 0; i <= n; i++) {
        push(&s, i + 1);
        if (i == n || pattern[i] == 'I') {
            while (!isEmpty(&s)) {
                printf("%d", pop(&s));
            }
        }
    }
}
```

```
        printf("\n");
    }

int main() {
    char pattern[MAX];
    printf("Enter pattern (containing only 'I' and 'D'): ");
    scanf("%s", pattern);
    printMinNumberForPattern(pattern);
    return 0;
}
```

Output:

Enter pattern (containing only 'I' and 'D'): IDID

Resulting minimum number: 21435

Question 8: Minimum Sum of Two Numbers Formed from Digits using Queue

Code

```
#include <stdio.h>

#define MAX 100

typedef struct {
    int arr[MAX];
    int front, rear;
} Queue;

void initQueue(Queue *q) {
    q->front = -1;
    q->rear = -1;
}

int isEmpty(Queue *q) {
    return (q->front == -1 || q->front > q->rear);
}

void enqueue(Queue *q, int val) {
    if (isEmpty(q)) {
        q->front = 0;
        q->rear = 0;
    } else {
        q->rear++;
    }
    q->arr[q->rear] = val;
}

int dequeue(Queue *q) {
    if (!isEmpty(q)) {
        int val = q->arr[q->front++];
        if (q->front > q->rear) {
            q->front = -1;
            q->rear = -1;
        }
        return val;
    }
    return -1;
}
```

```

}

void sortArray(int arr[], int n) {
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}

long findMinSum(int digits[], int n) {
    sortArray(digits, n);
    Queue q;
    initQueue(&q);
    for (int i = 0; i < n; i++) {
        enqueue(&q, digits[i]);
    }
    long num1 = 0, num2 = 0;
    int turn = 0;
    while (!isEmpty(&q)) {
        int digit = dequeue(&q);
        if (turn % 2 == 0) {
            num1 = num1 * 10 + digit;
        } else {
            num2 = num2 * 10 + digit;
        }
        turn++;
    }
    printf("Numbers formed: %lld and %lld\n", num1, num2);
    return num1 + num2;
}

```

```
int main() {  
  
    int n;  
  
    printf("Enter number of digits: ");  
  
    scanf("%d", &n);  
  
    int digits[MAX];  
  
    printf("Enter digits (0 to 9):\n");  
  
    for (int i = 0; i < n; i++) {  
        scanf("%d", &digits[i]);  
    }  
  
    long long min_sum = findMinSum(digits, n);  
  
    printf("Minimum Sum: %lld\n", min_sum);  
  
    return 0;  
}
```

Output:

Enter number of digits: 6

Enter digits (0 to 9):

6 8 4 5 2 3

Numbers formed: 246 and 358

Minimum Sum: 604

Question 9: First Negative Integer in Every Window of Size k using Queue

Code

```
#include <stdio.h>

#define MAX 100

typedef struct {
    int arr[MAX];
    int front, rear;
} Queue;

void initQueue(Queue *q) {
    q->front = -1;
    q->rear = -1;
}

int isEmpty(Queue *q) {
    return (q->front == -1 || q->front > q->rear);
}

void enqueue(Queue *q, int val) {
    if (isEmpty(q)) {
        q->front = 0;
        q->rear = 0;
    } else {
        q->rear++;
    }
    q->arr[q->rear] = val;
}

int dequeue(Queue *q) {
    if (!isEmpty(q)) {
        int val = q->arr[q->front++];
        if (q->front > q->rear) {
            q->front = -1;
            q->rear = -1;
        }
        return val;
    }
    return -1;
}
```

```

}

int getFront(Queue *q) {
    return q->arr[q->front];
}

void printFirstNegative(int arr[], int n, int k) {
    Queue q;
    initQueue(&q);
    for (int i = 0; i < k; i++) {
        if (arr[i] < 0) {
            enqueue(&q, i);
        }
    }
    for (int i = k; i <= n; i++) {
        if (!isEmpty(&q)) {
            printf("%d ", arr[getFront(&q)]);
        } else {
            printf("0 ");
        }
        while (!isEmpty(&q) && getFront(&q) <= (i - k)) {
            dequeue(&q);
        }
        if (i < n && arr[i] < 0) {
            enqueue(&q, i);
        }
    }
    printf("\n");
}

int main() {
    int n, k;
    printf("Enter number of elements: ");
    scanf("%d", &n);

    int arr[MAX];
    printf("Enter elements:\n");

```

```
for (int i = 0; i < n; i++) {  
    scanf("%d", &arr[i]);  
}  
printf("Enter window size k: ");  
scanf("%d", &k);  
printf("\nFirst negative integer in each window of size %d:\n", k);  
printFirstNegative(arr, n, k);  
return 0;  
}
```

Output:

Enter number of elements: 5

Enter elements:

-8 2 3 -6 1

Enter window size k: 2

First negative integer in each window of size 2:

-8 0 -6 -6

Question 10: Rotting Oranges (Time Required for All Oranges to Rot) using Queue

Code

```
#include <stdio.h>

#define MAX 100

typedef struct {
    int r, c, time;
} Element;

typedef struct {
    Element data[MAX * MAX];

    int front, rear;
} Queue;

void initQueue(Queue *q) {
    q->front = -1;
    q->rear = -1;
}

int isEmpty(Queue *q) {
    return (q->front == -1 || q->front > q->rear);
}

void enqueue(Queue *q, Element el) {
    if (isEmpty(q)) {
        q->front = 0;
        q->rear = 0;
    } else {
        q->rear++;
    }

    q->data[q->rear] = el;
}

Element dequeue(Queue *q) {
    Element el = q->data[q->front++];

    if (q->front > q->rear) {
        q->front = -1;
        q->rear = -1;
    }

    return el;
}
```

```
}
```

```
int orangesRotting(int grid[MAX][MAX], int m, int n) {  
    Queue q;  
    initQueue(&q);  
    int freshCount = 0;  
    for (int i = 0; i < m; i++) {  
        for (int j = 0; j < n; j++) {  
            if (grid[i][j] == 2) {  
                Element el = {i, j, 0};  
                enqueue(&q, el);  
            } else if (grid[i][j] == 1) {  
                freshCount++;  
            }  
        }  
    }  
    int maxTime = 0;  
    int dr[] = {-1, 1, 0, 0};  
    int dc[] = {0, 0, -1, 1};  
    while (!isEmpty(&q)) {  
        Element curr = dequeue(&q);  
        maxTime = curr.time;  
        for (int i = 0; i < 4; i++) {  
            int nr = curr.r + dr[i];  
            int nc = curr.c + dc[i];  
            if (nr >= 0 && nr < m && nc >= 0 && nc < n && grid[nr][nc] == 1) {  
                grid[nr][nc] = 2;  
                freshCount--;  
                Element next = {nr, nc, curr.time + 1};  
                enqueue(&q, next);  
            }  
        }  
    }  
}
```

```

        return (freshCount == 0) ? maxTime : -1;
    }
int main() {
    int m, n;
    printf("Enter matrix dimensions (rows cols): ");
    scanf("%d %d", &m, &n);
    int grid[MAX][MAX];
    printf("Enter matrix elements (0 for empty, 1 for fresh, 2 for rotten):\n");
    for (int i = 0; i < m; i++) {
        for (int j = 0; j < n; j++) {
            scanf("%d", &grid[i][j]);
        }
    }
    int result = orangesRotting(grid, m, n);
    if (result != -1) {
        printf("\nMinimum time required: %d\n", result);
    } else {
        printf("\nAll oranges cannot be rotten (-1)\n");
    }
    return 0;
}

```

Output:

Enter matrix dimensions (rows cols): 3 3

Enter matrix elements (0 for empty, 1 for fresh, 2 for rotten):

2 1 0

1 0 1

0 0 2

Minimum time required: 2